

概要设计说明书

项目名称：在线论文文档自动生成工具

版本：V2026.07

成员：	<u>42411051 李思贤</u>
成员：	<u>42411084 余锦标</u>
成员：	<u>42411094 谢宏涛</u>
成员：	<u>42411116 李佳蔚</u>
成员：	<u>42411134 张宗元</u>
成员：	<u>42411202 张熙伟</u>

实训课程软件工程文档

2026 年 7 月 15 日

目录

目录	1
1 系统总体架构	3
1.1 架构设计原则	3
1.2 B/S 三层架构	3
1.3 前后端分离交互流程	4
1.4 系统模块划分	4
2 技术选型	4
2.1 技术栈总览	4
2.2 后端技术选型分析	6
2.2.1 Spring Boot 4.1.0 + Java 21	6
2.2.2 Spring Data JPA + PostgreSQL 16	6
2.2.3 Redis 7	7
2.3 认证与安全方案	7
2.4 前端技术选型分析	8
2.4.1 Vue 3 + Vite	8
2.4.2 Axios + Vue Router	8
2.5 文档生成方案	8
3 数据库设计	8
3.1 数据库概述	8
3.2 实体关系设计	8
3.3 数据库表清单	9
3.4 核心表结构设计	9
3.4.1 用户表 (sys_user)	9
3.4.2 论文表 (thesis)	9
3.4.3 模板版本表 (template_version)	11
3.5 索引设计	11
4 开发环境与工程结构	12
4.1 开发环境配置	12
4.2 后端工程结构	12

版本修订记录

版本	日期	修改人	审核	修改说明
V1.0	2026-07-16	B	全组	初稿完成

1 系统总体架构

1.1 架构设计原则

本系统遵循以下架构设计原则：

- 1. **前后端分离**：前端采用 Vue 3 单页应用（SPA），后端采用 Spring Boot RESTful API，前后端通过 HTTP/JSON 协议通信，实现彻底的前后端解耦，便于独立开发、测试和部署；
- 2. **分层架构**：后端采用 Controller-Service-Repository 三层架构，各层职责明确、单向依赖，上层调用下层接口，下层不感知上层逻辑；
- 3. **无状态服务**：API 服务不持有会话状态，认证信息通过 JWT Token 在客户端和服务端之间传递，Token 缓存至 Redis 以保证分布式环境下的一致性；
- 4. **数据一致性**：核心业务数据采用关系型数据库 PostgreSQL 存储，利用外键约束、事务管理和数据库索引保证数据完整性和查询性能；
- 5. **缓存策略**：高频读取的热点数据（模板配置、用户会话）缓存在 Redis 中，减少数据库查询压力，提升系统响应速度。

1.2 B/S 三层架构

系统采用经典的 B/S（Browser/Server）三层架构模型，从逻辑上划分为表现层、业务逻辑层和数据访问层，如图 1 所示。



图 1: B/S 三层架构示意图

各层职责说明如下：

- **表现层**：负责用户界面展示和交互，基于 Vue 3 Composition API 构建组件化页面，使用 Element Plus 组件库统一 UI 风格，通过 Axios 封装 HTTP 请求与后端 API 通信。页面包括登录注册、论文编辑、模板管理、批阅审核等功能模块；

- **业务逻辑层**：作为系统核心，基于 Spring Boot 框架实现全部业务逻辑。Controller 层负责接收 HTTP 请求、参数校验和响应格式化；Service 层封装业务规则（用户认证、论文管理、模板控制、文档导出、批注批阅等）；Repository 层通过 Spring Data JPA 完成数据持久化操作；
- **数据访问层**：PostgreSQL 存储全部结构化业务数据（用户、论文、模板、批注等），Redis 提供 Token 缓存和热点数据缓存服务，服务器本地文件系统存储上传的图片资源和生成的 DOCX/PDF 文档文件。

1.3 前后端分离交互流程

前后端分离架构下的典型请求处理流程如下：

1. 用户在浏览器中操作，Vue 3 前端通过 Axios 发起 HTTP 请求（GET/POST/PUT/DELETE），请求头携带 JWT Token（格式：Authorization: Bearer <token>）；
2. 请求到达 Spring Boot 内嵌 Tomcat 服务器，首先经过 JWT 过滤器（JwtFilter）解析 Token 并将用户信息（userId、username、role）注入请求上下文；
3. 请求分发至对应的 Controller 方法，Spring MVC 完成参数绑定和 JSR-303 校验（@Valid）；
4. Controller 调用 Service 层执行业务逻辑，Service 通过 JPA Repository 与 PostgreSQL 数据库交互，必要时读写 Redis 缓存；
5. Service 将处理结果返回 Controller，Controller 通过统一返回体 Result 封装响应数据（code、message、data）返回前端；
6. 前端根据响应状态码和数据进行页面渲染或错误提示。

1.4 系统模块划分

系统按功能划分为六大核心模块，如表 1 所示：

六大模块的依赖关系为：M1 认证鉴权模块为所有模块提供统一的身份认证和权限校验基础；M2 模板论文模块是核心业务模块，M3 参考文献模块和 M4 导出模块依赖于 M2 的论文数据；M5 前端模块为 M1-M4 提供用户操作界面；M6 提交批阅模块串联学生、教师角色的协同工作流。

2 技术选型

2.1 技术栈总览

系统技术栈如表 2 所示，每一项技术的选择均基于明确的技术理由和项目约束。

表 1: 系统功能模块划分

模块编号	模块名称	功能说明
M1	认证与权限管理模块	用户注册、登录、登出, JWT Token 签发与验证, 角色权限控制 (ADMIN/STUDENT/TEACHER), 密码 BCrypt 加密存储
M2	模板与论文管理模块	管理员按学院管理论文模板 (封面字段、章节结构、格式参数), 支持模板版本管理; 学生创建论文、在线编辑章节内容 (富文本)、草稿自动保存与手动保存
M3	参考文献与图片管理模块	学生逐条录入参考文献信息 (支持 BibTeX 批量导入), 上传论文插图 (格式/大小校验), 图片内联展示
M4	文档导出模块	基于论文内容 + 模板格式配置, 服务端异步生成标准 DOCX (Apache POI) 和 PDF (LaTeX 编译) 文档, 提供下载
M5	前端交互与 UI 模块	Vue 3 组件化页面构建, 包括学生端 (论文编辑、预览、导出、提交)、教师端 (批注、评阅、退回)、管理员端 (模板管理、学院管理)
M6	提交批阅与缓存部署模块	学生论文提交/教师批阅/退回重提完整流程, Redis 缓存策略, 系统一键部署配置 (Docker Compose)

表 2: 系统技术栈总览

层次	技术	版本
后端框架	Spring Boot	4.1.0
编程语言	Java	21 (LTS)
Web 框架	Spring Web MVC (spring-boot-starter-webmvc)	4.1.0
ORM 框架	Spring Data JPA (Hibernate)	4.1.0
关系型数据库	PostgreSQL	16+
缓存中间件	Redis	7+
认证	JWT (io.jsonwebtoken)	0.12.6
密码加密	Spring Security Crypto (BCrypt)	6.x
API 文档	Knife4j (Swagger/OpenAPI 3)	4.5.0
前端框架	Vue 3 (Composition API)	3.x
构建工具	Vite	5.x
UI 组件库	Element Plus	2.x
HTTP 客户端	Axios	1.x
前端路由	Vue Router	4.x
DOCX 生成	Apache POI (poi-ooxml)	5.x
PDF 生成	LaTeX (xelatex) + 模板类	TeX Live 2024+
构建管理	Maven (mvnw wrapper)	3.9+

2.2 后端技术选型分析

2.2.1 Spring Boot 4.1.0 + Java 21

Spring Boot 4.1.0 是 Spring 生态最新的稳定主版本，基于 Java 21 LTS（长期支持版本）构建。选择理由如下：

- **约定优于配置：**自动配置机制大幅减少 XML 配置，通过 application.properties 管理全部参数，开发效率高；
- **内嵌 Tomcat：**无需单独部署 Web 容器，应用打包为可执行 JAR，配合 Maven Wrapper 实现零环境依赖启动；
- **生态成熟：**与 Spring Data JPA、Spring Web MVC、Spring Security Crypto 等子项目无缝集成；
- **Java 21 特性：**支持虚拟线程（Virtual Threads）、记录类（Records）、模式匹配（Pattern Matching）等现代语言特性，提升代码可读性和并发处理能力；
- **生产就绪：**内置 Actuator 健康检查、Metrics 监控端点，便于运维巡检。

2.2.2 Spring Data JPA + PostgreSQL 16

数据持久化层采用 JPA（Java Persistence API）规范实现，选择理由：

- **对象关系映射**: Java 实体类通过注解直接映射数据库表结构, 减少手写 SQL, 提高开发效率;
- **方法命名查询**: Spring Data JPA 通过 Repository 接口的方法命名规则自动生成查询 SQL, 常见 CRUD 操作无需编写 JPQL;
- **数据库迁移友好**: 通过 `spring.jpa.hibernate.ddl-auto=validate` 在校验模式下确保实体定义与数据库结构一致;
- **PostgreSQL 优势**: 支持 JSONB 类型 (模板配置的灵活存储)、丰富的索引类型 (B-tree、GIN)、ACID 事务保证、成熟的并发控制 (MVCC);
- **外键约束**: 利用数据库级外键保证引用完整性, 防止数据孤岛。

2.2.3 Redis 7

Redis 作为缓存层和 Token 存储, 选择理由:

- **高性能**: 纯内存操作, 读写延迟在毫秒级, 单机 QPS 可达 10 万 +;
- **数据结构丰富**: String 类型存储 Token 和序列化对象 (JSON), 支持设置过期时间 (TTL), 天然适合 JWT Token 的 24 小时过期管理;
- **轻量级**: 部署和维护成本低, 实训环境中单实例即可满足需求。

2.3 认证与安全方案

系统未引入 Spring Security 全套框架, 而是采用轻量级的自定义认证方案:

- **JWT Token (jjwt 0.12.6)**: 无状态 Token 认证, 服务端不保存会话, Token 自带用户 ID、用户名、角色信息, 通过 HMAC-SHA256 签名防篡改, 有效期 24 小时;
- **BCrypt 密码加密 (spring-security-crypto)**: 单向哈希加盐算法, 相同密码每次加密结果不同, 有效防御彩虹表攻击, work factor 默认 10 轮;
- **角色级权限控制**: 自定义 @RoleRequired 注解配合 Spring MVC 拦截器, 在 Controller 方法级别声明所需的角色 (ADMIN/STUDENT/TEACHER), 拦截器在请求进入 Controller 前校验当前用户角色是否在允许列表中;
- **前后端双重校验**: 前端路由守卫隐藏无权限页面入口, 后端 API 强制校验 JWT Token 和角色权限, 防止绕过前端直接调用 API。

2.4 前端技术选型分析

2.4.1 Vue 3 + Vite

Vue 3 采用 Composition API 编程范式,相比 Options API 具有更好的逻辑复用性和 TypeScript 支持; Vite 作为下一代前端构建工具,利用 ES 模块原生支持实现极速冷启动和热更新 (HMR),开发体验显著优于 Webpack。Element Plus 是基于 Vue 3 的成熟 UI 组件库,提供表单、表格、对话框、上传、富文本等丰富的企业级组件,降低前端开发工作量。

2.4.2 Axios + Vue Router

Axios 封装 HTTP 请求,统一配置 Base URL、请求超时、拦截器 (请求前自动注入 JWT Token,响应时统一处理 401 未授权跳转); Vue Router 4 实现前端单页路由,通过嵌套路由组织页面层级 (Layout 布局页 → 功能子页面),路由守卫在页面切换前校验用户登录状态和角色权限。

2.5 文档生成方案

文档导出是本系统的核心功能之一,采用两种技术路径:

- **DOCX 生成 (Apache POI):** Apache POI 是 Java 领域操作 Microsoft Office 文档的事实标准库,通过 poi-ooxml 模块可编程创建 DOCX 文件。服务端根据论文 HTML 内容解析文本段落、样式标记、图片引用,结合模板格式配置 (字体、字号、行距、页边距),按序构建 XWPFDocument 对象并输出 DOCX 字节流;
- **PDF 生成 (LaTeX + xelatex):** 利用已有的 LaTeX 模板类 (softeng.cls) 和编译流程,服务端将论文内容转换为 LaTeX 源码文件,调用 xelatex 编译器生成高质量 PDF 文档。LaTeX 排版引擎在数学公式、参考文献格式化、目录自动生成方面具有不可替代的优势。

3 数据库设计

3.1 数据库概述

系统数据库名称为 `thesis_generator`,使用 PostgreSQL 16 关系型数据库。数据库设计遵循第三范式 (3NF),核心实体间通过外键建立关联约束,保证数据引用完整性。共设计 10 张核心数据表和 1 张辅助历史记录表。

3.2 实体关系设计

系统的核心实体及其关系如下:

- **学院 (College) 与用户 (User):** 一对多关系。一个学院包含多名用户 (学生和教师),用户通过 `college_id` 外键关联所属学院;

- **学院 (College) 与模板 (Template):** 一对多关系。一个学院下可有多个论文模板, 模板通过 college_id 外键关联所属学院;
- **模板 (Template) 与模板版本 (TemplateVersion):** 一对多关系。一个模板可经历多次修订, 每次修订记录为一个模板版本, 模板版本通过 template_id 外键关联所属模板;
- **模板版本 (TemplateVersion) 与论文 (Thesis):** 一对多关系。一个模板版本可被多篇论文引用, 论文通过 template_version_id 外键关联模板版本, 同时记录所属学院;
- **用户 (User) 与论文 (Thesis):** 一对多关系。一名学生可创建多篇论文, 论文通过 student_id 外键关联学生用户;
- **论文 (Thesis) 与章节 (ThesisSection):** 一对多关系。一篇论文包含多个章节 (引言、各论文章节、结论、参考文献等), 章节通过 thesis_id 外键关联所属论文, 通过 parent_id 自引用实现章节嵌套;
- **论文 (Thesis) 与参考文献 (ThesisReference):** 一对多关系。一篇论文包含多条参考文献条目;
- **论文 (Thesis) 与图片 (ThesisImage):** 一对多关系。一篇论文中可嵌入多张图片;
- **论文 (Thesis) 与提交记录 (Submission):** 一对多关系。一篇论文可有多次提交记录 (学生修改后重新提交);
- **论文 (Thesis) 与批注 (Annotation):** 一对多关系。一篇论文可有教师添加的多条批注, 批注关联到具体章节和教师;
- **论文 (Thesis) 与批阅记录 (ReviewRecord):** 一对多关系。一篇论文可被多次批阅或退回。

3.3 数据库表清单

系统全部数据表及其职责如表 3 所示:

3.4 核心表结构设计

3.4.1 用户表 (sys_user)

用户表是系统权限体系的基础, 结构如表 4 所示:

3.4.2 论文表 (thesis)

论文表是系统核心业务数据表, 结构如表 5 所示:

论文状态流转规则为: DRAFT (编辑中) → COMPLETED (完成) → SUBMITTED (已提交) → REVIEWED (已批阅) / RETURNED (已退回, 回到 DRAFT)。

表 3: 数据库表清单

序号	表名	说明
1	sys_college	学院信息表, 存储学院名称和编号
2	sys_user	系统用户表, 存储全部用户账号、角色和所属学院
3	template	论文模板表, 存储模板基本信息和启用状态
4	template_version	模板版本表, 存储每个模板各版本的配置数据 (JSONB)
5	thesis	论文主表, 存储每篇论文的元信息
6	thesis_section	论文章节表, 存储章节内容和结构信息
7	thesis_reference	参考文献表, 存储论文的参考文献条目
8	thesis_image	图片资源表, 记录上传图片的文件信息
9	submission	提交记录表, 记录学生历次提交信息
10	annotation	批注表, 存储教师在论文指定位置的批注意见
11	review_record	批阅记录表, 存储教师的评语、分数和操作类型
12	thesis_section_draft_history	章节草稿历史表, 支持自动保存版本回退

表 4: sys_user 用户表结构

字段名	数据类型	约束	说明
id	BIGSERIAL	PK	自增主键
username	VARCHAR(50)	NOT NULL, UNIQUE	登录用户名
password	VARCHAR(255)	NOT NULL	BCrypt 加密密码
real_name	VARCHAR(50)	NOT NULL	用户真实姓名
role	VARCHAR(20)	NOT NULL, CHECK	ADMIN / STUDENT / TEACHER
college_id	BIGINT	FK → sys_college.id	所属学院
enabled	BOOLEAN	DEFAULT TRUE	账号启用状态
created_at	TIMESTAMP	DEFAULT NOW	创建时间
updated_at	TIMESTAMP	DEFAULT NOW	更新时间

表 5: thesis 论文表结构

字段名	数据类型	约束	说明
id	BIGSERIAL	PK	自增主
student_id	BIGINT	NOT NULL, FK	学生用户
template_version_id	BIGINT	FK	关联模板
title	VARCHAR(500)	NOT NULL	论文标
status	VARCHAR(20)	NOT NULL, CHECK	DRAFT/COMPLETED/SUBMITTED/REV
is_locked	BOOLEAN	DEFAULT FALSE	提交后锁
college_id	BIGINT	FK	所属学
created_at	TIMESTAMP	DEFAULT NOW	创建时
updated_at	TIMESTAMP	DEFAULT NOW	更新时

3.4.3 模板版本表 (template_version)

模板版本表利用 PostgreSQL JSONB 类型实现灵活的配置存储，结构如表 6 所示：

表 6: template_version 模板版本表结构

字段名	数据类型	约束	说明
id	BIGSERIAL	PK	自增主键
template_id	BIGINT	NOT NULL, FK	所属模板
version_number	VARCHAR(20)	NOT NULL	版本号（如 1.0、2.0）
is_current	BOOLEAN	DEFAULT TRUE	是否为当前版本
cover_fields	JSONB	NOT NULL	封面字段配置（JSON 数组）
chapter_structure	JSONB	NOT NULL	章节结构定义（JSON 数组）
format_config	JSONB	NOT NULL	排版格式参数（JSON 对象）
created_at	TIMESTAMP	DEFAULT NOW	创建时间

其中cover_fields 示例:["title", "author", "studentId", "college", "major", "advisor", "date"]
chapter_structure 示例指定章节层级和默认标题；format_config 包含字体、字号、行距、页边距等排版参数。

3.5 索引设计

为保障查询性能，系统在以下字段上建立了索引，如表 7 所示：

索引策略遵循以下原则：高频查询条件字段建索引（如 role、status），外键关联字段建索引（如 student_id、thesis_id、college_id），避免在频繁更新的字段（如 content）上建立索引。当前索引方案可覆盖系统的全部核心查询场景。

表 7: 数据库索引设计

索引名	表	字段	用途
idx_user_role	sys_user	role	按角色筛选用户
idx_template_college	template	college_id	按学院查询模板
idx_template_type	template	type	按类型筛选模板
idx_thesis_student	thesis	student_id	查询学生论文列表
idx_thesis_status	thesis	status	按状态筛选论文
idx_section_thesis	thesis_section	thesis_id	查询论文全部章节
idx_annotation_thesis	annotation	thesis_id	查询论文全部批注
idx_review_thesis	review_record	thesis_id	查询论文批阅记录

4 开发环境与工程结构

4.1 开发环境配置

表 8: 开发环境要求

工具/环境	版本要求	用途
JDK	21 LTS	Java 编译和运行
Maven	3.9+（已内置 mvnw）	项目构建与依赖管理
PostgreSQL	16+	关系型数据库
Redis	7+	缓存与 Token 存储
Node.js	18+	前端构建和开发服务器
TeX Live	2024+	LaTeX 文档编译（PDF 导出）
Git	2.40+	版本控制
IDE	IntelliJ IDEA / VS Code	代码开发

4.2 后端工程结构

后端工程遵循 Maven 标准目录结构，采用分包分层组织代码：

Listing 1: 后端工程目录结构

```
thesis-generator/  
+-- pom.xml                # Maven 配置（依赖/插件）  
+-- mvnw / mvnw.cmd        # Maven Wrapper（免安装）  
+-- src/main/java/com/example/thesisgenerator/  
|   +-- ThesisGeneratorApplication.java # Spring Boot 启动类  
|   +-- common/                # 通用组件  
|   |   +-- Result.java        # 统一 API 返回体  
|   |   +-- BusinessException.java # 业务异常类  
|   |   +-- GlobalExceptionHandler.java # 全局异常处理  
|   +-- config/                # 配置类
```

```
11 | | +-- CorsConfig.java          # CORS跨域配置
12 | | +-- RedisConfig.java        # Redis序列化配置
13 | | +-- JwtUtil.java           # JWT工具类
14 | | +-- JwtFilter.java         # JWT认证过滤器
15 | | +-- RoleRequired.java      # 角色权限注解
16 | | +-- RoleInterceptor.java   # 角色校验拦截器
17 | | +-- WebConfig.java         # MVC拦截器注册
18 | | +-- SwaggerConfig.java     # API文档配置
19 | +-- entity/                  # JPA实体类
20 | +-- repository/              # 数据访问层
21 | +-- dto/                     # 数据传输对象
22 | +-- service/                 # 业务逻辑层
23 | +-- controller/              # RESTful控制器
24 +-- src/main/resources/
25 | +-- application.properties   # 应用配置
26 | +-- schema.sql               # 数据库初始化脚本
27 +-- src/test/                  # 单元测试
```

该结构的优势为：代码按职责分层（controller/service/repository），再按业务模块分包，层次清晰、职责分明，新成员可快速定位代码位置。